

**Code of Conduct:**

I Abinavesh S P (19252146) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

**Signature of the student:**

**MARKS ALLOCATION**

|    | Problem Understanding (20%) | Method Selection (20%) | Solution Process (25%) | Accuracy of Calculation (20%) | Interpretation of Results (15%) |
|----|-----------------------------|------------------------|------------------------|-------------------------------|---------------------------------|
| Q1 | 1                           | 2                      | 2                      | 2                             | 1.5 8.5                         |
| Q2 | 1                           | 2                      | 2                      | 2                             | 1.5 8.5                         |
| Q3 | 2                           | 1                      | 1                      | 1                             | 2 7                             |
| Q4 | 2                           | 1                      | 1                      | 2                             | 1 7                             |
| Q5 | 1                           | 2                      | 2                      | 2                             | 1.5 8.5                         |

**RUBRICS FOR CALCULATION:**

| Criteria                  | Weightage | Level 4 - Exemplary                                                                  | Level 3 - Proficient                         | Level 2 - Developing                          | Level 1 - Beginning                              |
|---------------------------|-----------|--------------------------------------------------------------------------------------|----------------------------------------------|-----------------------------------------------|--------------------------------------------------|
| Problem Understanding     | 20%       | Clearly interprets problem, identifies all parameters and requirements accurately(4) | Understands problem with minor gaps(3)       | Partial understanding; misses key elements(2) | Misinterprets or unable to understand problem(1) |
| Method Selection          | 20%       | Selects most appropriate method/formula with strong justification(4)                 | Selects correct method with minor errors(3)  | Inappropriate or partially correct method(2)  | Unable to select method(1)                       |
| Solution Process          | 25%       | Logical, systematic steps with correct approach throughout(5)                        | Mostly correct steps with minor mistakes(4)  | Disorganized steps; multiple errors(3)        | No clear process(0)                              |
| Accuracy of Calculation   | 20%       | All calculations accurate; correct final answer(4)                                   | Minor calculation errors(3)                  | Frequent errors; incorrect final answer(2)    | No valid calculations(0)                         |
| Interpretation of Results | 15%       | Clearly interprets results with units and engineering relevance(3)                   | Basic interpretation with minor omissions(2) | Limited or unclear interpretation(1)          | No interpretation(0)                             |

39.5

**SIMATS ENGINEERING**  
**Department of Computer Science & Engineering**  
**ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING**

|                |                                                                                                                                                                                                                                   |               |                                                    |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|----------------------------------------------------|
| Course Code:   | CSA1223                                                                                                                                                                                                                           | Course Title: | Computer Architecture                              |
| CO Assessed:   | CO2                                                                                                                                                                                                                               | Assessment:   | ASSESSMENT TOOL1-<br>ANALYTICAL PROBLEM<br>SOLVING |
| Weightage:     | 45%                                                                                                                                                                                                                               | Total Marks:  | 100                                                |
| CO2 Statement: | Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3) |               |                                                    |
| Student Name:  | Abinavesh G.P                                                                                                                                                                                                                     | Reg.No.:      | 192521496                                          |
| Department:    | B.Tech IT                                                                                                                                                                                                                         | Date:         | 21-8-26                                            |

19%

**ANNEXURE A**

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g.,  $13 \div 3$ ) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

**SIMATS ENGINEERING**  
**Department of Computer Science & Engineering**  
**ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING**

---

|                       |                                                                                                                                                                                                                                   |                      |                                                             |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-------------------------------------------------------------|
| <b>Course Code:</b>   | <b>CSA1223</b>                                                                                                                                                                                                                    | <b>Course Title:</b> | <b>Computer Architecture</b>                                |
| <b>CO Assessed:</b>   | <b>CO2</b>                                                                                                                                                                                                                        | <b>Assessment:</b>   | <b>ASSESSMENT TOOL1-<br/>ANALYTICAL PROBLEM<br/>SOLVING</b> |
| <b>Weightage:</b>     | <b>45%</b>                                                                                                                                                                                                                        | <b>Total Marks:</b>  | <b>100</b>                                                  |
| <b>CO2 Statement:</b> | Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3) |                      |                                                             |
| <b>Student Name:</b>  |                                                                                                                                                                                                                                   | <b>Reg.No.:</b>      |                                                             |
| <b>Department:</b>    |                                                                                                                                                                                                                                   | <b>Date:</b>         |                                                             |

**ANNEXURE A**

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

## 1. Binary representation

+45

Convert 45 to binary:  $45 = 32 + 8 + 4 + 1$

$45_{10} = 00101101_2$     $+45 = 00101101$    **-23**

First represent +23:  $23_{10} = 00010111_2$

### Step 1: Complement all bits

$00010111 \rightarrow 11101000$

### Step 2: Add 1

$11101000 + 1 = 11101001$

Therefore,

$-23 = 11101001$

## 2. Addition: +45+(-23)

Align the two 8-bit operands: 00101101+11101001

0010110111101001100010110

000101102

Convert to decimal: 00010110=16+4+2=22, 45+(-23)=22

**Carry process** 010 010 110 001 110 101 001 110

## 3. Subtraction: +45-(-23)

In 2's complement, subtraction is performed by adding the **2's complement of the subtrahend**.

We have:  $45 - (-23) = 45 + 23$

The 8-bit representation of +23 is: 00010111

00101101+0001011101000100

Convert the result:

01000100=64+4=68

## 4. Sign-bit alignment

In an 8-bit signed number, the **MSB (Most Significant Bit)** is the sign bit:

- 0 → positive
- 1 → negative

-23=11101001

## 5. Overflow detection

For signed 2's complement addition, overflow occurs when:

- two **positive numbers** are added and the result becomes negative, or
- two **negative numbers** are added and the result becomes positive.

Another hardware method is:

Overflow=Cin to MSB  $\oplus$  Cout of MSB

where C represents carry.

### For +45+(-23)

The operands have **different signs**, so signed overflow **cannot occur**.

Result: 00010110=+22

No overflow

### For +45-(-23)

This becomes: +45+(+23)

Both operands are positive: 00101101+00010111=01000100

The result is also positive and equals +68.

No

overflow

## Results

| Operation        | Binary Result | Decimal Result | Overflow |
|------------------|---------------|----------------|----------|
| <b>+45+(-23)</b> | 00010110      | +22            | No       |
| <b>+45-(-23)</b> | 01000100      | +68            | No       |

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Answer:

Consider two 4-bit inputs:  $A=A_3 A_2 A_1 A_0$ ,  $B=B_3 B_2 B_1 B_0$

**Generate:**  $G_i = A_i B_i$

**Propagate:**  $P_i = A_i \oplus B_i$

A carry is propagated when exactly one of the input bits is 1.

The carry equation is:  $C_{i+1} = G_i + P_i C_i$

## 2. Carry Calculation in CLA

For a 4-bit CLA, instead of waiting for each carry, all carries are expanded in terms of the initial carry  $C_0$ .

### Carry C1

$$C1 = G0 + P0 C0$$

### Carry C2

$$C2 = G1 + P1 G0 + P1 P0 C0$$

### Carry C3

$$C3 = G2 + P2 G1 + P2 P1 G0 + P2 P1 P0 C0$$

### Carry C4

$$C4 = G3 + P3 G2 + P3 P2 G1 + P3 P2 P1 G0 + P3 P2 P1 P0 C0$$

## 3. Sum Calculation

$$Si = Pi \oplus Ci$$

$$S0 = P0 \oplus C0 \quad S1 = P1 \oplus C1 \quad S2 = P2 \oplus C2 \quad S3 = P3 \oplus C3$$

and C4 becomes the final carry-out.

| Bit | Ai | Bi | Pi = Ai<br>$\oplus$ Bi | Gi = Ai Bi |
|-----|----|----|------------------------|------------|
| 0   | 1  | 0  | 1                      | 0          |
| 1   | 1  | 1  | 0                      | 1          |
| 2   | 0  | 1  | 1                      | 0          |
| 3   | 1  | 0  | 1                      | 0          |

Now calculate carries:

$$C1 = G0 + P0 C0 = 0 + 1(0) = 0 \quad C2 = G1 + P1 G0 + P1 P0 C0 = 1 \quad C3 = G2 + P2 G1 + P2 P1 G0 + \dots \\ C3 = 0 + (1)(1) = 1 \quad C4 = G3 + P3 G2 + P3 P2 G1 + \dots = 1$$

The sum is: 1011+0110=10001

$$A+B=100012$$

## 5. CLA vs Ripple Carry Adder

### Ripple Carry Adder

In an RCA, each full adder must wait for the carry from the previous stage:

$$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4$$

$$TRCA \approx nT_{\text{carry}}$$

For a 4-bit RCA:

$$TRCA \approx 4T_{\text{carry}}$$

## Carry Look-Ahead Adder

$$C_1, C_2, C_3, C_4$$

Approximate delay:

$$TCLA \approx T_{\text{generate}} + T_{\text{logic}} + T_{\text{sum}}$$

This is significantly smaller than the ripple delay for larger word sizes.

## 6. Why CLA Is Faster

The major advantage of CLA is that it **eliminates the sequential carry dependency**.

In an RCA:

$$\text{Carry}_1 \rightarrow \text{Carry}_2 \rightarrow \text{Carry}_3 \rightarrow \text{Carry}_4$$

In a CLA:

All carries are computed from  $P_i, G_i, C_0$

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers,  $-6$  and  $+5$ . Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Soln:

### BINARY REPRESENTATION

Using 4-bit 2's complement:

$$+6 = 0110$$

To find  $-6$ :

$$0110 \rightarrow 1001 \rightarrow 1010$$

$$-6 = 1010 + 5 = 0101$$

Initial values:

$$\text{Accumulator (A)} = 0000$$

$$\text{Multiplier (Q)} = 0101$$

$$\text{Multiplicand (M)} = 1010$$

$$\text{Booth bit (Q-1)} = 0$$

Since the operands are 4-bit, 4 iterations are performed.

## 2. BOOTH'S DECISION RULES

Q0 Q-1 Operation

00 No operation

$$01 A = A + M$$

$$10 A = A - M$$

11 No operation

## 3. STEP-BY-STEP EXECUTION

$$\text{Initial: } A = 0000 \text{ } Q = 0101 \text{ } Q-1 = 0 \text{ } M = 1010$$

$$\text{Iteration 1: } Q_0 Q_{-1} = 10 \text{ } A = A - M \text{ } 0000 - 1010 = 0110$$

Arithmetic Right Shift:

$$0110 \ 0101 \ 0$$

↓

$$0011 \ 0010 \ 1$$

$$A = 0011$$

$$Q = 0010$$

$$Q_{-1} = 1$$

$$\text{Iteration 2: } Q_0 Q_{-1} = 01 \text{ } A = A + M \text{ } 0011 + 1010 = 1101$$

Arithmetic Right Shift:

$$1101 \ 0010 \ 1$$

↓

1110 1001 0

$A = 1110$   $Q = 1001$   $Q-1 = 0$

Iteration 3:

$Q_0 Q_{-1} = 10$   $A = A - M$   $1110 - 1010 = 0100$

Arithmetic Right Shift:

0100 1001 0

↓

0010 0100 1

$A = 0010$   $Q = 0100$   $Q-1 = 1$

Iteration 4:

$Q_0 Q_{-1} = 01$

$A = A + M$   $0010 + 1010 = 1100$

Arithmetic Right Shift:

1100 0100 1

↓

1110 0010 0

$A = 1110$   $Q = 0010$

#### 4. FINAL RESULT

Combine A and Q:  $AQ = 1110\ 0010$

Product =  $11100010_2$

$11100010 \rightarrow 00011101 \rightarrow 00011110$

$00011110_2 = 30_{10}$

$11100010_2 = -30_{10}$

$(-6) \times (+5) = -30$

FINAL ANSWER:

$(-6) \times (+5) = 11100010_2 = -30_{10}$

4. A processor is required to perform division of two binary numbers (e.g.,  $13 \div 3$ ) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

## 1. Restoring Division Algorithm

### Step-by-step

| Step | After Shift (A,Q) | A – M                | Action     | Final A | Q    |
|------|-------------------|----------------------|------------|---------|------|
| 1    | 00001 1010        | $00001 - 00011 = -2$ | Restore    | 00001   | 1010 |
| 2    | 00011 0100        | $00011 - 00011 = 0$  | No restore | 00000   | 0101 |
| 3    | 00000 1010        | $00000 - 00011 = -3$ | Restore    | 00000   | 1010 |
| 4    | 00001 0100        | $00001 - 00011 = -2$ | Restore    | 00001   | 0100 |

Quotient =  $0100_2 = 4$

Remainder =  $00001_2 = 1$

$13 \div 3 = 4$  remainder 1

## 2. Non-Restoring Division Algorithm

A = 00000 Q = 1101 M = 00011

Rules:

- If  $A \geq 0$ :  $A = A - M$
- If  $A < 0$ :  $A = A + M$
- Set  $Q_0 = 1$  if the resulting  $A \geq 0$ ; otherwise  $Q_0 = 0$ .
- Perform the next shift.

### Step-by-step

| Step | After Shift A | Operation | Result A     | Q    |
|------|---------------|-----------|--------------|------|
| 1    | 00001         | $A - M$   | $11110 = -2$ | 1010 |
| 2    | $11101 = -3$  | $A + M$   | 00000        | 0101 |
| 3    | 00000         | $A - M$   | $11101 = -3$ | 1010 |
| 4    | $11011 = -5$  | $A + M$   | $11110 = -2$ | 0100 |

At the end, A is negative, so perform the final correction:

$A = A + M$

$11110 + 00011 = 00001$

Therefore:

$$\text{Quotient} = 0100_2 = 4$$

$$\text{Remainder} = 00001_2 = 1$$

$$13 \div 3 = 4 \text{ remainder } 1$$

### 3. Comparison

| Feature         | Restoring Division                             | Non-Restoring Division                      |
|-----------------|------------------------------------------------|---------------------------------------------|
| Restoration     | Required whenever A is negative                | Avoided during intermediate steps           |
| Operations      | Subtraction + possible restoration             | Addition or subtraction                     |
| Number of steps | n iterations + possible restoration operations | n iterations + at most one final correction |
| Hardware        | More complex                                   | Simpler and faster                          |
| Speed           | Lower                                          | Higher                                      |
| Efficiency      | Lower due to extra restoration                 | Higher                                      |

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g.,  $-13.25$ ), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

#### 1. Conversion of $-13.25$ to Binary

$$\text{Integer part: } 13_{10} = 1101_2 \quad \text{Fractional part: } 0.25_{10} = 0.01_2$$

$$13.25_{10} = 1101.01_2$$

$$\text{Normalized form: } -13.25 = -1.10101 \times 2^3$$

#### 2. Single Precision – 32 Bits

$$\text{Single precision contains: Sign} = 1 \text{ bit} \quad \text{Exponent} = 8 \text{ bits} \quad \text{Mantissa} = 23 \text{ bits} \quad \text{Bias} = 127$$

$$\text{Since the number is negative: } S = 1$$

$$\text{Actual exponent: } E = 3 + 127 = 130 = 10000010_2$$

The leading 1 is implicit, so the mantissa is:

$$10101000000000000000000$$

Hence:

1 | 10000010 | 101010000000000000000000

### 3. Double Precision – 64 Bits

Double precision contains:

Sign = 1 bit

Exponent = 11 bits

Mantissa = 52 bits

Bias = 1023

Sign:

$$S = 1$$

Exponent:

$$E = 3 + 1023 = 1026 = 100000000010_2$$

Mantissa:

101010000000000000000000000000000000000000000000000000000

$1 \mid 10000000010 \mid 1010100000000000000000000000000000000000000000000$

#### 4. Floating-Point Addition

When two floating-point numbers are added, the processor follows these steps:

### Step 1: Exponent Alignment

The exponents are compared. The mantissa of the number with the smaller exponent is shifted right until both exponents are equal.

### Step 2: Mantissa Addition

The aligned mantissas are added if the signs are the same. If the signs are different, the smaller mantissa is subtracted from the larger one.

### Step 3: Normalization

The result is converted into normalized form:

$$1.\text{xxxxx} \times 2^n$$

The exponent is adjusted accordingly.

### Step 4: Rounding

If the result contains more bits than the available mantissa, the extra bits are rounded off. IEEE 754 normally uses round-to-nearest, ties-to-even.

### Step 5: Final Representation

| Criteria | Weightage | Level 4 - Exemplary | Level 3 - Proficient | Level 2 - Developing | Level 1 - Beginning |
|----------|-----------|---------------------|----------------------|----------------------|---------------------|
|----------|-----------|---------------------|----------------------|----------------------|---------------------|

|                           |     |                                                                                      |                                              |                                               |                                                  |
|---------------------------|-----|--------------------------------------------------------------------------------------|----------------------------------------------|-----------------------------------------------|--------------------------------------------------|
| Problem Understanding     | 20% | Clearly interprets problem, identifies all parameters and requirements accurately(4) | Understands problem with minor gaps(3)       | Partial understanding; misses key elements(2) | Misinterprets or unable to understand problem(1) |
| Method Selection          | 20% | Selects most appropriate method/formula with strong justification(4)                 | Selects correct method with minor errors(3)  | Inappropriate or partially correct method(2)  | Unable to select method(1)                       |
| Solution Process          | 25% | Logical, systematic steps with correct approach throughout(5)                        | Mostly correct steps with minor mistakes(4)  | Disorganized steps; multiple errors(3)        | No clear process(0)                              |
| Accuracy of Calculation   | 20% | All calculations accurate; correct final answer(4)                                   | Minor calculation errors(3)                  | Frequent errors; incorrect final answer(2)    | No valid calculations(0)                         |
| Interpretation of Results | 15% | Clearly interprets results with units and engineering relevance(3)                   | Basic interpretation with minor omissions(2) | Limited or unclear interpretation(1)          | No interpretation(0)                             |